Cube Library for Android

Version 1.00

Table of contents

1 Introduction ... 2
2 Prerequisites ... 3
3 ViewModel - LiveData ... 4
4 ViewModel - Functions ... 5
4.1 General ... 5
4.2 License ... 6
4.3 Scanning for devices ... 7
4.4 Connecting a device ... 8
4.5 Device commands ... 9
5 MessageData ... 10
5.1 MessageData ... 10
5.2 Message types ... 10
5.3 MT_INFO - Message codes ... 11
5.4 MT_ERROR - Message codes ... 12
5.5 Command errors ... 13
5.6 Device errors ... 15
6 List of device/interface states ... 18
7 Additional data types ... 19
7.1 DeviceInfo ... 19
7.2 DeviceInfoExt ... 19
7.3 DateTime ... 19
7.4 PowerState ... 19
7.5 PublicLicenseData ... 19
7.6 MeasurementData ... 20
7.7 ResultData ... 21
7.8 LineData ... 22
8 Communication/state overview ... 23
Version History ... 24

1 Introduction

The Cube Library for Android provides an interface for communications with a Chembio Diagnostics GmbH Cube/Microreader I device. This document describes this interface and its usage as well as some general information.

The interface supports BluetoothLE connections as well as connections through a USB-cable (with OTG-Adapter).

For usage examples, please refer to the DataReader app provided by Chembio Diagnostics GmbH.

All code samples in this document and the example app are written in the Kotlin programming language.

2 Prerequisites

To use the library, copy the provided aar-file to your projects libs-folder and add it to your project dependencies.

Additionally needed dependencies are:

- implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:2.5.1"
- implementation "androidx.lifecycle:lifecycle-livedata-ktx:2.5.1"
- implementation "androidx.lifecycle:lifecycle-common-java8:2.5.1"
- implementation "androidx.room:room-ktx:2.5.0"
- kapt "androidx.room:room-compiler:2.5.0"

Version numbers are those used at the creation of this document and can change over time.

All BluetoothLE communication is handled through the BLESSED library, which needs to be included in your project as well:

```
allprojects {
	repositories {
		maven { url 'https://jitpack.io' }
	}
}

dependencies {
	implementation "com.github.weliem:blessed-android-coroutines:$version"
}
```

3 ViewModel - LiveData

The library is set up in the form of a ViewModel, exposing observeable LiveData objects and trigger functions. There should always be only one ViewModel, shared by all fragments needing access.

The ViewModel needs the application as parameter in its constructor (both BluetoothLE and USB connections require this to work).

The following list contains all observeable LiveData-Objects published by the ViewModel.

Name	Type	Description
state	EState	Represents the current state of the library. See list of states for details on the single states.
devices	List<DeviceInfo>	List of devices available for connection – update by calling startScan()
lastMessage	MessageData	Messages are used to propagate all kinds of events. See chapter Messages for detailed information.
connectedDeviceInfoExt	DeviceInfoExt	Contains additional information for the connected device. Will be invalidated on disconnecting.
measurements	List<MeasurementData>	List of all measurements in the librarys database. Measurements read from the connected device (either through a measurement or by reading the device database) will be automatically added.
selectedMeasurement	MeasurementData	The currently selected measurement. Controls content of measurementLineData and measurementResults. Can be changed through selectMeasurement().
measurementResults	List<ResultData>	Contains a list of all result data linked to the current selectedMeasurement.
measurementLineData	List<LineData>	Contains a list of all line data linked to the current selectedMeasurement.

4 ViewModel - Functions

4.1 General

getVersion(): String

Returns a string containing the version of the library.

setAutoDisconnectTime(seconds: Long)

Set the time (in seconds) after which a device will be automatically disconnected when in state ST_IDLE. Set to 0 to disable auto-disconnect.

clearLocalDatabase()

Delete all entries in the local (library) database. Does not affect any connected device.

selectMeasurement(measurementData: MeasurementData)

Try to select the given measurement. Will update the following LiveData-Objects:

- selectedMeasurement
- measurementResults
- measurementLineData.

4.2 License

Connecting a device will require a license, which should be provided with the library (addinitional file). An invalid or missing license will lead to failure of all connection attempts.

setLicense(dataStream: InputStream): Boolean

Try setting a license with data transferred through the stream. A license file will be provided with the library.
Returns true if license successfully set and valid, else false.

licenseValid(): Boolean

Returns the validity of the current set license.
Will return false if:

- no license is set
- the license is invalid
- the license is expired

getLicenseData(): PublicLicenseData

Return information on currently set license (date of expiration). Will return invalid data if no license is set.

4.3 Scanning for devices

startScan(timeOut_ms: Long = 5000): Boolean

Start a new scan for devices.

A running scan will be stopped when:

- the timeout is reached (only applicable if timeout > 0)
- the function connectDevice() is being called
- the function stopScan() is being called

All found devices will be added to the **devices**-LiveData-Object.

stopScan()

Stop a running scan. Will have no effect if no scan is running.

isScanning(): Boolean

Returns true if a scan is currently under way and false if not.

4.4 Connecting a device

isConnected(): Boolean

Returns true if a device is currently connected, and false if not.

connectDevice(deviceInfo: DeviceInfo, disableButton: Boolean): Boolean

Try to connect to the given device.
If “disableButton” is true, the device button will be disable while the connection is active.
While the connection is being established, the state will be set to ST_CONNECT. A successful connection is indicated by the state changing to ST_IDLE and an IM_CONNECTION_ESTABLISHED info message.
If the connection can not be established, the state will change back to ST_DISCONNECTED.

disconnectDevice(finishCurrentOperation: Boolean, shutDown: Boolean)

Disconnects from the currently connected device. If device button was disabled on connect, it will be reenabled.
If “finishCurrentOperation” is true, any currently running operation will be finished and the disconnect is executed as soon as the state ST_IDLE is reached.
If “shutdown” is true, the device will be shut down (not supported by all devices).
While disconnecting, the state will change to ST_DISCONNECT. After the disconnect, the state will be set to ST_DISCONNECTED and **lastMessage** to IM_DEVICE_DISCONNECTED.
Please be aware that the list of available devices will be cleared after a disconnect, so you will have to scan again to establish another connection.

4.5 Device commands

startEvaluation(withTimer: Boolean, configStream: InputStream? = null): Boolean

Attempts to initialize a measurement on the connected device. Returns true if measurement could be initialized, else false.
Param "withTimer" controls wether an instant measurement is executed (false) or the timer specified in the test configuration is counted down before the measurement is executed.
Param "configStream" is optional and allows to provide a test configuration for the measurement. If no test configuration is provided, the device will try to read it from a RFID tag.
During the evaluation process, the state will be set to ST_EVALUATE.

abortEvaluation(): Boolean

Try to abort the current measurement – will only work if the measurement timer is running, as soon as the actual measurement is executed, abortion is not possible anymore. Returns true if measurement could be aborted, else false.

startReadDeviceDatabase(): Boolean

Initialize a full read of the device database, therefore collecting all measurements currently saved on the device. Returns true if reading could be initialized.
During the read process, the state will be ST_READ, and progress will be relayed through **lastMessage**.

formatDeviceDatabase(): Boolean

Format the device database, deleting all saved measurement data. Returns true if formatting could be initialized. Successful formatting will be indicated by IM_DB_FORMATTED.

5 MessageData

5.1 MessageData

```
data class MessageData(
        val msgType: EMessageType = EMessageType.MT_NO_MESSAGE,
        val msgCode: Int = 0,
        val msgData: Int = 0,
        val msgID: Long = System.currentTimeMillis()
)
```

The msgID is there to enable useful comparisons (e.g., check if that message has already been handled).

5.2 Message types

The following table describes the possible values of **EMessageType** (parameter msgType):

Message type	Value	Message code	Description
MT_NO_MESSAGE	0x00	Invalid	No actual message (initial value)
MT_INFO	0x01	See **list of InfoMessages**	Informational messages
MT_ERROR	0x02	See **list of ErrorMessages**	Error messages should always be handled, can lead to the termination of an existing connection.
MT_PROGRESS	0x03	**Message code** will contain the current progress (as absolute number), while **Data** represents the maximum progress value.	Progress update (e.g., when reading device database). Should **Data** (maximum progress) have the value 0, this message is to be interpreted as a busy signal from the interface.

5.3 MT_INFO - Message codes

The following table describes the potential values fort he parameter **msgCode** in case of **msgType** being **MT_INFO**:

InfoMessage	Value	msgData	Description
IM_NO_INFO_MESSAGE	0x00		Not an info message
IM_CONNECTION_ESTABLISHED	0x01		A device has been connected.
IM_DEVICE_DISCONNECTED	0x02		A device has been disconnected.
IM_PLACE_WHITE	0x03		Evaluation: Put the white cassette (undeveloped test) in the device and press the device button.
IM_PLACE_TEST	0x04		Evaluation: Put the test cassette (developed test) in the device and press the device button.
IM_TIMER_RUNNING	0x05	Seconds left until the execution of the measurement	Evaluation: Measurement timer is running (will be sent approximately once per second as long as the timer of a measurement is running).
IM_EVALUATION_RUNNING	0x06		Evaluation: Measurement evaluation is running (will be sent once the actual measurement starts).
IM_EVALUATION_READ_DATA	0x07		Evaluation: Reading result data for measurement.
IM_MEASUREMENT_DONE	0x08		Will be sent whenever a measurement has been executed successfully.
IM_MEASURE_COUNT	0x09	The number of measurements read from the device	The database has been read.
IM_DB_FORMATTED	0x0A		Database successfully formatted.

5.4 MT_ERROR - Message codes

The following table describes the potential values for the parameter **msgCode** in case of **msgType** being **MT_ERROR**:

ErrorMessage	Value	msgData	Description
EM_NO_ERROR_MESSAGE	0x00		No valid error message
EM_DEVICE_ERROR	0x01	See **list of device errors**	A device error occurred (see list of device errors).
EM_COMMAND_ERROR	0x02	See **list of command errors**	A command error occured (see list of command errors).
EM_TIME_OUT	0x03		Time out on device communication. Will be followed by either idle-state (if communication could be reinstated) or disconnect (if not).
EM_LICENSE_MISMATCH	0x04		License is either invalid or does not match the device.
EM_NO_VALID_LICENSE_SET	0x05		There is no license set.
EM_BLUETOOTH_CONNECTION_FAILED	0x06		Connection via Bluetooth LE has failed.
EM_PERMISSION_MISSING	0x07		Not all needed permissions have been granted.
EM_BLUETOOTH_DISABLED	0x08		Bluetooth is disabled.
EM_LOCATION_SERVICES_DISABLED	0x09		Location services arte disabled. These are needed to scan for Bluetooth scanning in some Android versions.

5.5 Command errors

List of **communication errors**:

Error	Value	Description
DC_ERR_NO_ERROR	0x00	no error (placeholder, won't result in an actual message)
DC_ERR_CRC_MISMATCH	0x01	CRC check failed (device received a data package with invalid CRC)
DC_ERR_UNKNOWN_COMMAND	0x02	the command is not supported by this device
DC_ERR_RECEIVE_BUFFER_OVERFLOW	0x04	the received package is to big to fit in the buffer
DC_ERR_NOT_ENOUGH_MEMORY	0x05	the device does not have enough free memory to handle the received package
DC_ERR_COMMAND_ERROR	0x06	placeholder - occurs if a command is not yet completely implemented (was used in development phase)
DC_ERR_INVALID_FLASH_START_ADDRESS	0x10	flash start address is invalid
DC_ERR_INVALID_FLASH_END_ADDRESS	0x11	flash end address is invalid
DC_ERR_INVALID_EEPROM_START_ADDRESS	0x12	eeprom start address is invalid
DC_ERR_INVALID_EEPROM_END_ADDRESS	0x13	eeprom end address is invalid
DC_ERR_PAGE_WRITE_ERROR	0x14	error while writing in flash memory
DC_ERR_UNKNOWN_SUB_COMMAND	0x21	sub command is not implemented
DC_ERR_NOT_POWERED	0x24	device is not powered
DC_ERR_FLASH_IS_WRITE_PROTECTED	0x25	flash is write protected
DC_ERR_FLASH_WRITE_FAILED	0x26	write check failed (memory needs to be erased before being written)
DC_ERR_IMAGE_NO_DATA	0x30	there is no available image data for transmission
DC_ERR_IMAGE_INVALID_FLAGS	0x31	the command flags for the image data request are invalid
DC_ERR_WRITE_BIG_BUFFER_OVERFLOW	0x40	the package size is to big to fit in the buffer
DC_ERR_WRITE_BIG_BUFFER_INVALID_ADDRESS	0x41	the target address for the write operation is invalid (no memory assigned)

Error	Value	Description
DC_ERR_READ_BIG_BUFFER_NO_FKT	0x42	the target address for the write operation is invalid (no write function for this memory location)
DC_ERR_READ_BIG_BUFFER_NO_DATA	0x43	attempted to read more data than is available (with auto size)
DC_ERR_WRITE_BIG_MEMORY_OVERFLOW	0x44	attempted to write more data than the memory can contain
DC_ERR_WRITE_BIG_FLASH_FAILED	0x45	writing in the flash memory failed
DC_ERR_READ_BIG_MEMORY_OVERFLOW	0x46	attempted to read more data than is available at the memory location
DC_ERR_CUBE_NOT_VALIDATED	0x50	the command is not supported without validation

5.6 Device errors

List of **device errors**:

Error	Value	Description
NO_ERROR	0x00	Placeholder, won't result in an actual message
ERR_RESTART	0x01	
ERR_EEPROM_CORRUPTED	0x03	EEPROM corrupted
ERR_SENSOR_CORRUPTED	0x04	Sensor corrupted
ERR_LOW_BATT	0x05	Low battery
ERR_INTERNAL_CONFIG_CORRUPTED	0x06	Internal test configuration corrupted
ERR_CUBE_EXPIRED_LOCK	0x08	Device locked due to expiration
ERR_CUBE_EXPIRED_NOT_LOCKED	0x0A	Device expired, but not locked
ERR_RTC_CORRUPTED	0x0B	RTC corrupted
ERR_EXTERNAL_SRAM_CORRUPTED	0x0C	External SRAM corrupted
ERR_LAST_SAVED_RESULT_CORRUPTED	0x0D	Last saved result corrupted
ERR_WRITING_CHECKSUM_FAIL	0x0E	Checksum refresh failed
ERR_INTERNAL_FLASH_CORRUPTED	0x0F	Internal flash corrupted
EVAL_EXPIRY_DATE_EXCEEDED	0x10	The expiry date of the test configuration has been exceeded
EVAL_LOW_BATTERY	0x11	Voltage going below minimum value during measurement
EVAL_CAPTURE_BLACK_ERROR_HIGH	0x12	Errors on image capturing
EVAL_CAPTURE_BLACK_ERROR_LOW	0x13	
EVAL_CAPTURE_LIGHT_ERROR_HIGH	0x14	
EVAL_CAPTURE_LIGHT_ERROR_LOW	0x15	
EVAL_NOMINAL_POS_OUT_OF_RANGE	0x16	Nominal position of a testline is out of range
EVAL_SEARCH_WIDHT_SMALLER_LINE_WIDTH	0x17	Search width of a test line is smaller than its actual width
EVAL_INVALID_CONFIGURATION	0x1A	Loaded test configuration is invalid
EVAL_INVALID_CONFIG_VERSION	0x1C	Loaded test configuration has an invalid/unsupported version
EVAL_USER_ABORTED	0x1D	Evaluation has been aborted by the user
EVAL_QC_TEST_NEEDED	0x1E	A QC test is needed before another measurement can be done
RES_ASSIGN_ILLEGAL_INDEX	0x20	Result assignment: illegal index
RES_ASSIGN_ILLEGAL_CHARACTER	0x21	Result assignment: illegal character
RES_ASSIGN_ILLEGAL_JUMP	0x22	Result assignment: illegal jump address
RES_ASSIGN_ILLEGAL_CHAR_IN_NUMBER	0x23	Result assignment: expected number but found char
RES_ASSIGN_ILLEGAL_INDEX_2	0x24	Result assignment: illegal index
RES_ASSIGN_ILLEGAL_INDEX_3	0x25	Result assignment: illegal index
RES_ASSIGN_INDEX_OUT_OF_RANGE	0x26	Result assignment: index out of range

Error	Value	Description
RES_ASSIGN_VALUE_LOW	0x28	Result assignment: value out of range (below range)
RES_ASSIGN_VALUE_HIGH	0x29	Result assignment: value out of range (above range)
RES_ASSIGN_EXPECTED_OPERAND	0x2A	Result assignment: expected operand, received something else
RES_ASSIGN_JUMP_ADDRESS_OUT_OF_RANGE	0x2B	Result assignment: jump address out of range
RES_ASSIGN_ILLEGAL_ASSIGNMENT	0x2C	Result assignment: assignment not valid (can not reassign constant values)
RES_ASSIGN_OPERAND_IS_NAN	0x2D	Result assignment: assigned constant value is invalid (not a number)
RES_ASSIGN_INFINITE_LOOP	0x2E	Result assignment: potential infinite loop
RFID_TIME_OUT	0x30	time out on receiving data from RFID
RFID_STATUS_ERROR	0x31	RFID status error
RFID_RECEIVE_LENGTH_OVERFLOW	0x32	buffer to small for received data
RFID_COLLISION_DETECTED	0x33	communication collision detected (more than one tag in range)
RFID_MIFARE_NAK	0x34	MIFARE-tag: acknowledge has not been received
RFID_RECEIVE_CRC_INVALIDE	0x35	the CRC for the received data is invalid
RFID_BUFFER_TO_SMALL	0x36	buffer to small for requested operation
RFID_ATQA_ANSWER_INVALID	0x37	ATQA-command failed
RFID_NOTHING_RECEIVED	0x38	timeout on receiving RFID data (no tag in range)
RFID_STATUS_INVALID	0x39	RFID: status invalid
RFID_INTERNAL_ERROR	0x3A	RFID: internal error
RFID_MEMORY_ADDRESS_OUT_OF_RANGE	0x3B	the requested address is not available in the RFID memory (out of range)
RFID_MEMORY_AUTHENTIFICATION_FAILED	0x3C	the authentification of a RFID memory block failed
RFID_TRANCEIVE_ERROR	0x3D	error while transceiving data
RFID_READ_BUFFER_TO_SMALL	0x3E	the read buffer is to small for requested operation
RFID_NO_VALID_TAG	0x3F	no RFID tag detected or initialization failed
RFID_MAIN_TIMEOUT	0x40	timeout on RFID communication
RFID_MEMORY_ADDRESS_OUT_OF_RANGE2	0x41	the requested address is not available in the RFID memory (out of range)

Error	Value	Description
TEST_CONFIGURATION_GETCHAR_TIMEOUT	0x45	timeout during read operation
TEST_CONFIGURATION_GETCHAR_EOF	0x46	end of file reached (more data requested than available)
TEST_CONFIGURATION_CFG_TO_BIG	0x47	test configuration exceeds maximum size
TEST_CONFIGURATION_UNKNOWN_CFG	0x48	test configuration unknown (version not supported)
TEST_CONFIGURATION_EMPTY	0x49	test configuration empty
TEST_CONFIGURATION_FLASH_FULL	0x4A	(saving results) test configuration does not fit in remaining memory
TEST_CONFIGURATION_CHECKSUMFAIL	0x4B	test configuration: checksum mismatch (test configuration corrupted)
TEST_NO_CONFIG_FOUND	0x4C	no test configuration found
DB_LOAD_REQUESTED_RESULT_CFG_NOTFOUND	0x50	loading result: corresponding configuration not found
DB_LOAD_RESULT_EMPTY	0x51	loading result: result empty
DB_LOAD_RESULT_DELETED	0x52	loading result: result deleted
INT_FLASH_ERASE_FAIL	0x63	erasing/nulling of internal flash failed
RFID_WRITE_ENCRYPT_FIRST_MUST_START_AT_0	0x70	writing encrypted tag for the first time: start at address 0
RFID_UNENCRYPTED_TAG_READ_NOT_ALLOWED	0x71	reading of unencrypted RFID tags is not allowed
RFID_CONFIG_NO_CREDIT	0x72	the credit counter on the RFID tag has reached 0, the tag is not usable anymore
RFID_NOT_SUPPORTED	0x73	the RFID tag type is not supported
CRYPT_ALU_NOT_ENOUGH_DATA	0x80	validation errors
CRYPT_ALU_CALCULATION_PROGRAMMED_BREAK	0x81	
CRYPT_ALU_INTERNAL_PROGRAM_EMPTY	0x82	
CRYPT_RANDOM_PATTERN_INVALID	0x83	
TEST_CONFIGURATION_NO_VALID_CUSTOMER	0x91	the test configuration is not meant for the connected device
SAVE_TEST_NO_RESULT_FOUND_WITH_THIS_ID	0x9E	the given ID does not match any saved result

6 List of device/interface states

Name	Value	Description
ST_DISCONNECTED	0x00	No active device connection
ST_SCAN	0x10	Scanning for devices
ST_CONNECT	0x20	A connection is being established
ST_DISCONNECT	0x30	An active connection isa being terminated
ST_IDLE	0x40	A device is connected and ready for operation
ST_DEVICE_BUSY	0x50	The connected device is busy with another operation (e.g., measurement has been started through device button)
ST_ERROR_ACKNOWLEDGE	0x60	A timeout/error occurred during device communication, trying to restore the communication
ST_READ	0x70	Data is being read from the device
ST_EVALUATE	0x80	A measurement is being run on the connected device
ST_FORMAT_DEVICE_DB	0x90	The device database is being formatted

7 Additional data types

7.1 DeviceInfo

Contains name and connection type (FTID or BTLE) for the device.

7.2 DeviceInfoExt

Contains firmware and serial strings for the connected device as well as device date/time and information on battery/power supply (battery/power not applicable for all devices).
Device date/time and battery info are updated once per second when state is ST_IDLE.

7.3 DateTime

Simple structure to hold date and time values. Gives access to year, month, day, hour, minute and second.

7.4 PowerState

Gives information on battery voltage, external power supply etc.

7.5 PublicLicenseData

Provides additional license information (date of expiration).

7.6 MeasurementData

Represents an entry in the measurements table of the library database:
Enables access to:

- uid: String
  The unique identifier of this measurement.
- deviceSerialNumber: String
  The serial number of the device that has taken the measurement.
- dateTime: DateTime
  The date and time of the measurement.
- temperature: Float
  The temperature at the time of the measurement.
- imageData: Array<Int>
  Array of image values for the measurement, the size of the array equals the width of the image. The height of the image is always one pixel. Can be empty if device is not configured to save image data.
- cfgName: String
  The name of the test configuration used for the measurement.
- cfgLotNr: String
  The LOT-number of the test configuration used for the measurement.
- cfgManufacturerDistributor: String
  The manufacturer/distributor of the test as specified in the test configuration.

7.7 ResultData

Represents an entry in the results table of the library database.
Enables access to:

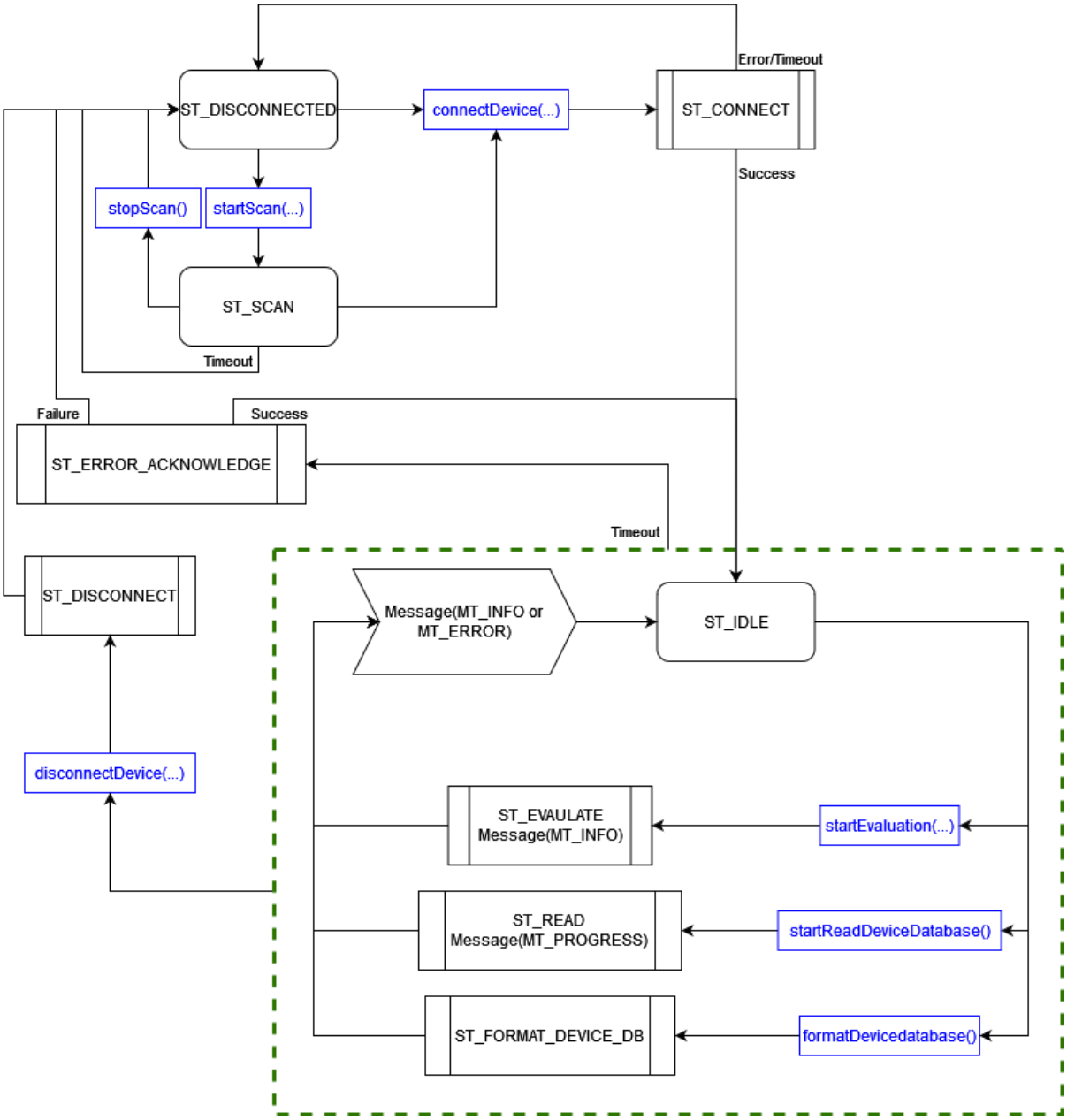
- uid: String
  The unique identifier of this result.
- measureLink: String
  The unique identifier of the measurement this result belongs to.
- idx: Int
  The index of this result.
- validity: Boolean
  Will be set to false if the result is not considered valid (e.g., if the control line has not been found).
- name: String
  The name of the result (as defined in the test configuration).
- resultValueFormatted: String
  The result value (same as displayedby the device).
- unit: String
  The unit of the result (as defined in the test configuration).
- resultClass: String
  The class of the result (as displayed by the device).

7.8 LineData

Represents an entry in the line data table of the library database. Can be empty if device is not configured to save line data.
Enables access to:

- uid: String
  The unique identifier of this line.
- measureLink: String
  The unique identifier of the measurement this line belongs to.
- idx: Int
  The index of this line.
- mean: Float
  The mean value of this line.
- min: Float
  The minimum value of this line.
- max: Float
  The maximum value of this line.
- leftMean: Float
  The mean value of the left half of the line.
- rightMean: Float
  The mean value of the right half of the line.
- usedPixel: Int
  The number of used (valid) pixels for the line.
- maxPos: Int
  The position of the maximum value for the line.
- centerPos: Int
  The center position of the line.
- width: Int
  The width of the line.
- fluorescence: Boolean
  Indicates if the line was defined as a fluorescence line.

8 Communication/state overview



Version History

Version	Changes	Date
1.00	Initial release based on Cube Android Library Version 0.1.20	February 2023